Developer Operating Manual
(Antigravity IDE + Modified Superpowers)

Purpose: A repeatable, disciplined workflow to build any new application with high code quality and low token waste.

1. Your Setup (What You Have)

1.1 Global Rules

These files are always-on behavioral constraints and are the foundation of consistent outcomes:

- AGENTS.md — cross-tool constitution: plan before coding, evidence-first debugging, stay in scope, lock architecture early, no destructive actions without approval.
- GEMINI.md — Antigravity-specific behavior: Fast vs Planning mode, token conservation, workflow/skill integration, memory discipline, and explicit Sequential Thinking usage.
- .antigravityignore – contains folders and files which should be ignored by LLM models

1.1.1 Project Specific Rules

- .graphifyignore – contains folders and files which should be ignored in graph (as of now it is a copy of antigravityignore)

1.2 Global Workflows (Slash Commands)

Primary phase entry points you use daily:

- /brainstorming /plan, /tdd, /debug, /review, /document, /refactor, /optimize, /hotfix, /sprint, /onboard, /close-session, /new-knowledge-item

Internal workflows exist under global_workflows/internal/ and are supporting wrappers (not daily entry points).

1.3 Global Skills

Canonical daily skills (small, stable) used by workflows:

- feature-planning
- tdd-workflow
- systematic-debugging
- code-review
- verification-before-completion
- refactoring
- security-review

- performance-audit
- api-design (optional)

Internal skills (skills/internal/) are intentionally strong or meta-level: using-superpowers, writing-skills, writing-plans, test-driven-development (strict reference).

1.4 MCPs (Model Context Protocol)

Your setup uses MCPs to reduce hallucination and improve reasoning without dumping large context into prompts:

- Sequential Thinking — structured planning (2–3 architecture options with tradeoffs) and debugging (hypothesis tracking).
- Context7 — documentation injection to confirm API usage before implementing unknown APIs.

```
{
  "mcpServers": {
    "sequential-thinking": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-sequential-thinking"
      ],
      "env": {}
    },
    "context7": {
      "command": "npx",
      "args": [
        "-y",
        "@upstash/context7-mcp"
      ]
    }
  }
}
```

1.5 Graphify (Persistent Codebase Understanding)

Graphify builds a project knowledge graph to avoid re-reading the repository in every session and to accelerate onboarding and architecture questions.

One-time setup
pip install graphifyy
graphify install --platform gemini

Per-project usage (from project root)

graphify .

Then in Antigravity chat
/graphify query "What are the main architectural patterns?"

Incremental Update graph after development (Recommended — fastest)
graphify . --update

2. Visual Summary (How Work Flows)

3. The Operating Loop (Step-by-Step for Any New Application)

Step 0 — Frame the Task (You/ Human-first, 5–10 minutes)

1. Write 2–6 sentences: problem statement, in-scope, out-of-scope, constraints, success criteria.
2. If you cannot write this clearly, you must brainstorm first.

Step 1 — Brainstorm (Only When Needed)

Use brainstorming ONLY when requirements are unclear, risky, or broad. Do not use it for trivial edits or Fast Mode tasks.

- Ask one clarifying question at a time.
- Produce: problem statement, in-scope/out-of-scope, constraints, success criteria.
- Stop once clarity is achieved; proceed to /plan.

Step 2 — /plan (Mandatory for Non-trivial Work)

3. Run /plan in a dedicated planning chat.
4. Use Sequential Thinking to propose 2–3 architecture options with tradeoffs.
5. Select ONE architecture, document rationale.
6. Produce artifacts: implementation_plan.md and task_checklist.md.
7. STOP and wait for approval before coding.

Step 3 — Implement (Small Loops)

Pick one task from task_checklist.md at a time. Choose the right mode:

- Fast Mode: single-file edits, docs, style fixes, mechanical refactors.
- /tdd: any behavior change or bug fix (tests fail first, minimal pass, then refactor).
- Parallel agents: only for independent tasks; avoid multiple agents editing same files.

Step 4 — /debug (When Something Breaks)

- Evidence first: reproduce, logs/stack traces, isolate failure point.
- Sequential Thinking: list hypotheses, test one at a time, one change at a time.
- If 3 fix cycles fail: STOP and escalate to architectural review.

Step 5 — /review (Quality Gate)

- Review changes against plan, correctness, security, maintainability, performance, and tests.
- Produce a findings table + verdict: Approve / Approve with comments / Request changes.
- Fix Critical/Major issues before continuing.

Step 6 — /document (Only If Docs Must Change)

- Update README / API docs / onboarding / architecture notes / changelog when behavior or setup changes.
- STOP and present docs for review before committing.

Step 7 — /close-session (Always)

- Update SUMMARY.md (intent, completed, decisions, open items, next steps, artifacts).
- Run KI gate: create a Knowledge Item only for durable decisions/constraints/root causes/invariants/tradeoffs.
- Stop cleanly; do not start new work inside close-session.

4. Model Selection: Right Model for the Right Task (Token Reduction)

Use the least powerful model that can do the task reliably. Escalate only when needed.

4.1 Three-tier strategy

- Tier 1 (Fast): mechanical edits, scaffolding, simple docs, small isolated changes.
- Tier 2 (Standard): multi-file implementation, integration, typical debugging.
- Tier 3 (Most capable): architecture decisions, deep debugging after repeated failures, security review, complex refactors.

4.2 Workflow-to-model mapping (recommended)

- /brainstorming → Tier 2
- /plan → Tier 3
- /tdd → Tier 1 (Tier 1 if truly trivial)
- /debug → Tier 2 (escalate to Tier 3 after repeated failures)
- /review / security-review → Tier 2
- /document → Tier 1
- /optimize → Tier 2
- /close-session → Tier 1

4.3 Token discipline checklist

- Avoid re-reading unchanged files; use surgical edits.
- Batch similar changes; avoid narration.
- Use Context7 for unknown APIs to avoid rework.
- Use Graphify for architecture questions instead of reading large folders repeatedly.
- Stop immediately when the task is complete.

5. Graphify: How It Fits and When To Use It

- Run Graphify when the repo grows beyond ~20 files, you restart sessions often, or you keep asking "where is X?"
- Use Graphify outputs to answer architecture questions quickly and reduce repeated context.
- Re-run after major milestones or significant refactors.

6. MCPs: How To Use Them Effectively

Sequential Thinking

- Planning: generate 2–3 architectures, compare tradeoffs, pick one.
- Debugging: hypothesis list, one test at a time, one change at a time.

Context7

- Use before implementing unfamiliar APIs.
- Use when errors suggest a version mismatch or outdated assumptions.

7. Worked Example (Generic, Any App)

Scenario: Build a small service with a domain model, persistence interface, and a simple API. The point is the process, not the stack.

Example Steps

8. Frame: Write a short problem statement and explicit scope/out-of-scope.
9. If unclear: brainstorm to clarify success criteria and constraints.
10. Plan: /plan → choose one architecture and produce implementation_plan.md + task_checklist.md.
11. Implement: /tdd each behavior unit; use Fast Mode for scaffolding.
12. Debug: /debug if tests fail; hypothesis-driven fixes.
13. Review: /review before finishing the milestone.
14. Close: /close-session → SUMMARY.md; create Knowledge Item only if KI gate passes.

9. Where to Put This Manual

- Keep this document as a reference for every new project.
- Optionally also publish as a GitHub portal (docs/ + MkDocs) for navigation and search.